Fundamentals of EC2

Explore AWS EC2 fundamentals focusing on leveraging Amazon Machine Images, instance types, and lifecycle states to build and manage scalable, secure cloud applications. Understand how to optimize performance, automate deployments, and monitor environments using EC2 features.

Amazon Elastic Compute Cloud (EC2) delivers resizable compute capacity in the cloud. At its core, EC2 allows us to launch virtual servers, instances, based on predefined Amazon Machine Images (AMIs) templates. Each instance provides virtualized CPU, memory, storage, and networking resources, configurable through instance types tailored for different workload demands.

Although Amazon EC2 is typically associated with systems administration and infrastructure management, it plays a crucial role in modern software development workflows. Developers frequently use EC2 to:

- Host backend microservices and RESTful APIs.
- Automatically execute, build, and test scripts as part of a CI/CD pipeline.
- Provide controlled environments for integration, system, and performance testing.
- Run containers (via ECS or self-managed Docker setups).
- Simulate production-like environments locally using isolated, testable VMs.

EC2 integrates seamlessly with developer-centric AWS services like CodeDeploy, Systems Manager, and CloudWatch, allowing us to easily automate deploy observability, and rollback strategies.

Ask

Amazon Machine Images (AMIs)

AMIs are the blueprint for launching EC2 instances. They encapsulate the software stack, operating system, and configurations required to spin up a new instance. AMIs are fundamental in creating repeatable, consistent environments across development, staging, and production.

There are three core categories of AMIs:

- **Public AMIs:** They are offered by AWS or third parties and are suitable for quick prototypes. These images often require post-launch configuration and are best suited for testing or short-term environments.
- **Custom AMIs:** These are created from a running EC2 instance and include the entire state of that instance's OS, installed packages, configurations, and data. They are ideal for scaling, automation, and ensuring consistent deployments.
- **Private AMIs:** These are not publicly shared and are controlled within an AWS account. They help enforce internal compliance and governance rules, particularly when licensed software or proprietary setups are involved.

We can also configure the operating system, region, architecture (32-bit or 64-bit), launch permissions, and root storage device type (EBS or instance store) in an AMI. Understanding these distinctions helps developers to architect reliable and optimized EC2 environments.

Ask

Creating a custom AMI

For developers aiming to build and deploy applications on AWS efficiently and reliably, learning to create and use custom AMIs is critical. A **custom AMI** is a personalized server template: the application code, all necessary libraries, specific runtime environments, and crucial configurations are prebaked and ready to launch. This directly translates into significantly faster instance boot times, rock-solid consistency across all environments, from development through to production, and a significant reduction in complex, error-prone bootstrapping scripts at launch.

Let's look at an example to create a custom AMI from a configured instance:

main.sh Shell-20

```shell
aws ec2 create-image \
  --instance-id <valid_instance_id> \
  --name "custom-app-server-v1" \
  --description "Base AMI with Python and NGINX" \
  --no-reboot
```

Create a custom AMI

This command captures the current state of an EC2 instance including OS, packages, and configurations and makes it reusable across our environment while optionally skipping the reboot phase.

Ask

AMI lifecycle

We can create, manage, or delete our AMIs. We can create different types of AMIs, as we discussed above, such as Amazon EBS or instance store-backed. We can also modify the AWS Marketplace AMI to suit our needs and register it to launch multiple instances from it.

An AMI progresses through various stages:

- **Create:** Use an EC2 instance as a base and configuration, with snapshots and metadata included.
- **Modify:** Edit metadata (description, permissions). Binary content is immutable.
- **Share:** Privately share with AWS accounts/organizations or make public (with caution).
- **Deprecate:** Disable future instance launches. It warns users without deleting the AMI.
- **Deregister:** Remove from the registry; this does not affect the existing instances.

The illustration below shows how AMI transitions from one stage to the other sequentially:

AMI lifecycle

Ask

Lifecycle awareness allows developers to automate version control, deprecate outdated templates, and enforce deployment governance.

Sharing AMIs

Sharing AMIs helps standardize development environments across teams. Developers can:

- Use `modify-image-attribute` to share AMIs with specific account IDs.
- Share with the entire AWS organization via Resource Access Manager (RAM).
- Make AMIs public, though this requires strong security to avoid leaking sensitive data.

The AWS CLI command below shows how we can share an AMI with another account:

main.sh Shell-20

```
aws ec2 modify-image-attribute \
  --image-id <valid_instance_id> \
  --launch-permission "Add=[{UserId=<valid_account_ID>}]"
```

Share an AMI with another account

This approach is often used for internal platform teams distributing standardized base images to application teams.

Ask

EC2 instance types and families

Every EC2 instance is launched from an AMI and must be tied to an instance type, which defines its virtual hardware. These instance types are grouped into families according to workload profiles, and each offers a unique combination of compute, memory, storage, and network throughput capabilities.

Instance Family	Optimization	Use Cases
`t`, `t4g`	Burstable, cost-efficient	Microservices, low-traffic web servers
`m`	Balanced	General-purpose backend APIs, enterprise workloads
`c`	Compute-intensive	Batch processing, data science, and video encoding
`r`, `x`, `z`	Memory-intensive	In-memory DBs, real-time analytics, and ERP systems
`i`, `d`, `h`	Storage-intensive	NoSQL DBs, log analytics, and Hadoop
`p`, `g`, `inf`, `f`	Accelerated computing	Machine learning, graphics rendering
`mac`	macOS environments	iOS/macOS app build and test

It is essential to understand how to choose the right instance family based on performance goals, budget, and workload needs for infrastructure design and deployment.

Instance lifecycle and behavior

An EC2 instance behaves like a virtual server that transitions through multiple lifecycle states. Understanding the behavior of instances is important for a developer to manage stateful applications, orchestrate workflows, and confidently implement automation.

Let's take a deeper look at the different states of EC2 instances.

Ask

- **Pending:** The instance is initializing. This is where user data scripts are typically executed.
- **Running:** The instance is live and operational. Monitoring and scaling tools like CloudWatch are activated.
- **Stopping/Stopped:** Once the instance is running, we can stop and start the instance again anytime. It is important to note that when an instance is stopped, the data stored in the RAM is lost, and the instance may not have the same public IP, but the private IP stays the same. The instance store volumes are also lost; AWS offers an alternative to preserve the data stored in the RAM using hibernation. Hibernation saves the contents of RAM to the EBS root volume. It also preserves the other EBS volumes attached. During reboot, the contents of RAM saved earlier are again loaded into memory.
- **Shutting down/Terminated:** Once the work is done and we no longer require the instance, we can terminate it. Terminating an instance changes its state from “Running” to “Shutting down.” The instance cannot be started again after it has been terminated.

Ask

Instance lifecycle

Advanced control mechanisms such as Auto Scaling lifecycle hooks, Systems Manager Run Command, and shutdown scripts provide rich automation opportunities at each stage of the lifecycle.

> **Note:** Hibernation is especially noteworthy as it saves the instance's RAM state to disk and restores it on restart, preserving session memory and speeding up boot times in certain use cases.

Performance and cost optimization

Developers can choose from several compute strategies in EC2 based on how predictable, scalable, or cost-sensitive their workloads are:

Compute Option	Description	Best For
On-Demand	You pay-as-you-go with flexible pricing and you can launch or terminate at any time.	It is ideal for short-lived apps, testing, and unpredictable workloads.
Reserved Instances (RI)	You prepay for one or three years, receiving a deep discount for steady-state workloads.	Use this option for always-on services like backend APIs or internal web tools.
Spot Instances	You bid on unused capacity, potentially saving up to 90%. However, these instances can be interrupted by AWS.	They are best suited for batch processing, fault-tolerant jobs, and auto-scaled containerized workloads.
Savings Plans	You receive a flexible discount based on compute usage, and the discount is not tied to a specific instance type.	Choose this for applications that span instance families or use services like Fargate and Lambda.
Dedicated Hosts	Physical servers are assigned exclusively to your account.	They are recommended for license-bound workloads (e.g., BYOL for Oracle or SQL Server), and scenarios that require strict compliance.
Capacity Reservations	You are guaranteed availability in a specific AZ for a particular	Use this option for high-availability systems that need guaranteed compute ca-

Ask

	instance type.	pacity during peak hours.

Optimizing EC2 usage is a combination of architectural choices and proactive management.

Developer-oriented use cases for EC2

EC2 provides powerful compute capabilities for various developer tasks, such as:

- **Continuous integration and build automation:** EC2 can run Jenkins or GitLab runners for compiling, testing, and packaging applications.
- **API hosting:** Developers often deploy stateless microservices or server-side GraphQL resolvers to EC2, particularly when more control or GPU access is needed compared to serverless models.
- **Custom environments for development:** When specific OS-level customization or legacy dependencies are required, EC2 offers full control over the stack.
- **Large-scale testing:** Load tests, chaos engineering experiments, or integration tests across distributed systems are well-suited for EC2-based simulations.
- **AI/ML inference:** Developers can use GPU-powered EC2 instances (like `g4`, `p4`) to serve trained models at scale for real-time predictions.

Monitoring and governance

Monitoring EC2 is essential in secure environments. Here are the AWS services commonly used for this purpose:

- **CloudTrail:** It captures API calls like `CreateImage`, `RegisterImage`, and `ModifyImageAttribute`.

Ask

- **EventBridge rules:** It automates workflows (e.g., trigger an alert when an AMI is shared publicly).
- **AWS Config:** It tracks compliance and is especially useful for auditing AMIs in regulated sectors.

Conclusion

In this lesson, we explored the essential elements of Amazon EC2 relevant for developers managing scalable, high-performing applications in the cloud. We covered how AMIs serve as the building blocks for repeatable infrastructure, how to navigate instance types and their optimal use cases, how lifecycle stages align

Ask